

Asistente Educativo Inteligente Basado en ESP32 e Inteligencia Artificial para la Respuesta de Consultas

Mijaíl André Agüero Cangalaya, José Antonio Casachahua Peña, Ccorahua Quispe Sthif, Alejandro Coronado Guiorzo

Universidad Nacional Federico Villarreal

RESUMEN

Este documento aborda la segunda fase de ingeniería (Entregable E2) del Asistente Educativo Inteligente. Se expone el diseño interno del firmware a través de una Máquina de Estados Finitos (FSM) no bloqueante. Asimismo, se detalla el mapa de memoria del microcontrolador ESP32 bajo una arquitectura Harvard modificada, justificando la partición de la memoria IRAM y DRAM para el manejo del Acceso Directo a Memoria (DMA). Se incluye el código fuente comentado desde la perspectiva arquitectónica, las interfaces de comunicación configuradas a bajo nivel y una proyección técnica de la evidencia de funcionamiento, finalizando con la demostración teórica del paradigma de programa almacenado frente al manejo de interrupciones de hardware.

I. INTRODUCCIÓN

La integración de Modelos de Lenguaje Grande (LLMs) en entornos académicos ha demostrado reducir significativamente la curva de aprendizaje en disciplinas técnicas. Sin embargo, la dependencia de interfaces humano-máquina (HMI) basadas en texto impone restricciones ergonómicas y cognitivas. En laboratorios, talleres o entornos de educación inclusiva, teclear una consulta interrumpe el flujo de trabajo y excluye a usuarios con discapacidades visuales o motrices.

Este proyecto resuelve dicha problemática mediante el desarrollo de un "Asistente Educativo

Inteligente" basado en tecnología *Edge IoT* (Internet de las Cosas en el Borde). El dispositivo actúa como un tutor físico "manos libres", capaz de capturar la voz del estudiante, orquestar la inferencia en la nube y sintetizar una respuesta acústica.

El paso de la conceptualización (Entregable E1) al diseño detallado (Entregable E2) exige resolver desafíos críticos de arquitectura de computadores. A diferencia de las computadoras de propósito general que delegan estas tareas a un sistema operativo complejo (como Linux o Windows), el uso del SoC ESP32 (arquitectura Xtensa LX6) obliga a gestionar manualmente los recursos del hardware. El procesamiento de audio digital en tiempo real consume un inmenso ancho de banda de memoria. Si el microcontrolador gestionara cada muestra de audio mediante su Unidad Lógica Aritmética (ALU), el sistema sufriría un colapso por inanición de recursos (*starvation*), perdiendo la conexión Wi-Fi y bloqueando el sistema.

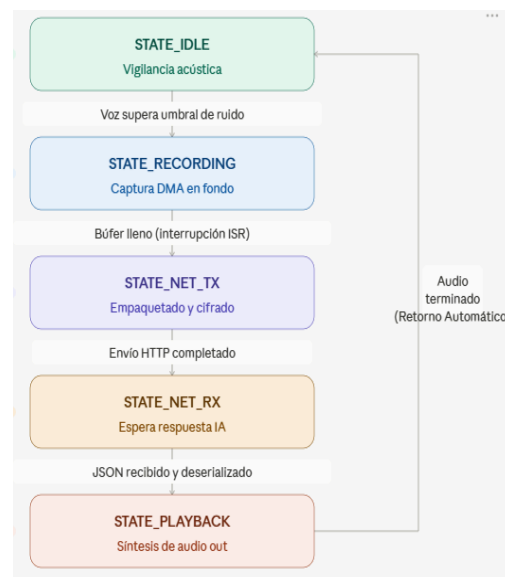
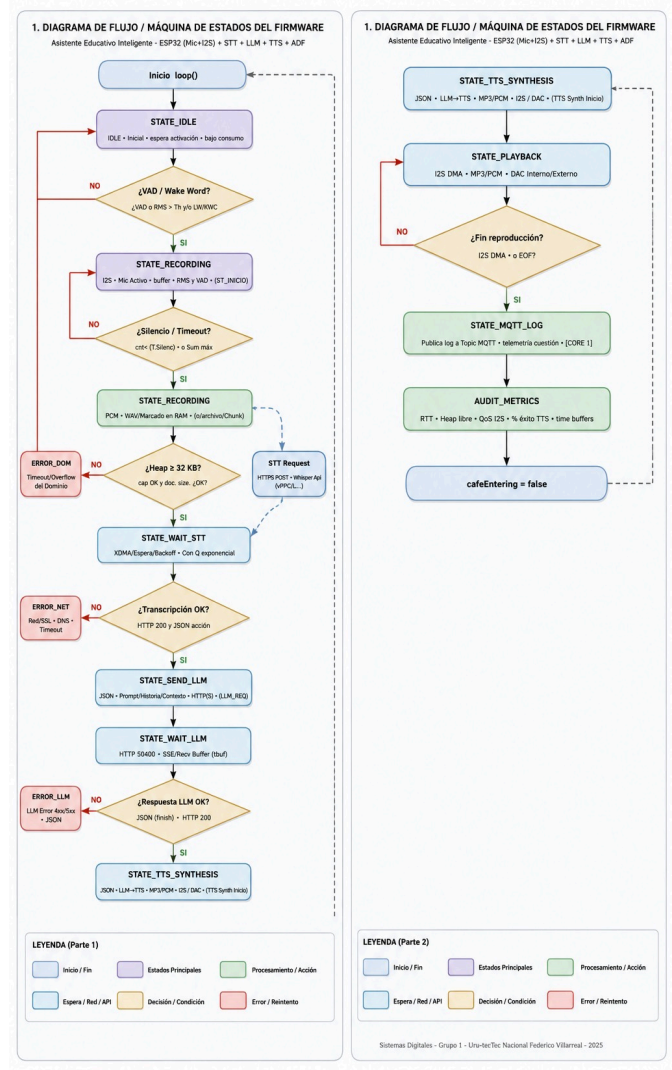
Por consiguiente, este documento detalla la implementación de soluciones a nivel de silicio: el uso de controladores de Acceso Directo a Memoria (DMA) para puentear a la CPU, la reubicación de código crítico en la caché de instrucciones (IRAM) para evitar latencias de lectura, y la estructuración del firmware mediante una Máquina de Estados Finitos (FSM) concurrente. Estos elementos demuestran la aplicación práctica de los fundamentos de la máquina de programa almacenado frente a entornos de alta exigencia transaccional.

II. MÁQUINA DE ESTADOS DEL FIRMWARE (DIAGRAMA DE FLUJO LÓGICO)

Para garantizar un determinismo estricto de tiempo real, el flujo del programa evita bucles de espera lógicos (como `delay()` o `while()`). En su lugar, el sistema operativo FreeRTOS administra una Máquina de Estados Finitos (FSM) regida por eventos e interrupciones.

- Estado STATE_IDLE (Vigilancia Acústica):** El procesador principal opera en estado de bajo consumo. El periférico I2S y el canal DMA asociado capturan audio en *buffers* circulares continuos. La CPU despierta periódicamente para ejecutar un algoritmo de Detección de Actividad de Voz (VAD). Si la energía de la señal no supera el umbral de ruido de fondo, el *buffer* se sobrescribe.
- Estado STATE_RECORDING (Adquisición Continua):** Al detectar el comando de voz del estudiante, la CPU bloquea la sobrescritura y encadena múltiples descriptores DMA para retener hasta 5 segundos de audio (aprox. 160 KB). Al completarse la ráfaga, el hardware DMA dispara una Interrupción (ISR).
- Estado STATE_NET_TX (Transmisión Criptográfica):** El "Core 0" asume el control. Empaqueta los datos de la memoria DRAM, aplica cifrado criptográfico (TLS 1.2/1.3) y abre un *socket* TCP/IP hacia la API REST del servicio de Inteligencia Artificial.
- Estado STATE_NET_RX (Recepción y Deserialización):** El ESP32 cede ciclos de reloj al planificador del sistema mientras espera el *payload* de respuesta. Al recibir la trama JSON, la CPU la deserializa, extrae el archivo de audio codificado de la respuesta educativa y lo mapea en el *Heap*.
- Estado STATE_PLAYBACK (Síntesis y Retorno):** El flujo de control se invierte. El DMA toma los datos del *Heap* y los empuja eléctricamente hacia el bus I2S de salida (amplificador). Al vaciarse el último descriptor, se genera una

interrupción de limpieza y la FSM retorna a STATE_IDLE.



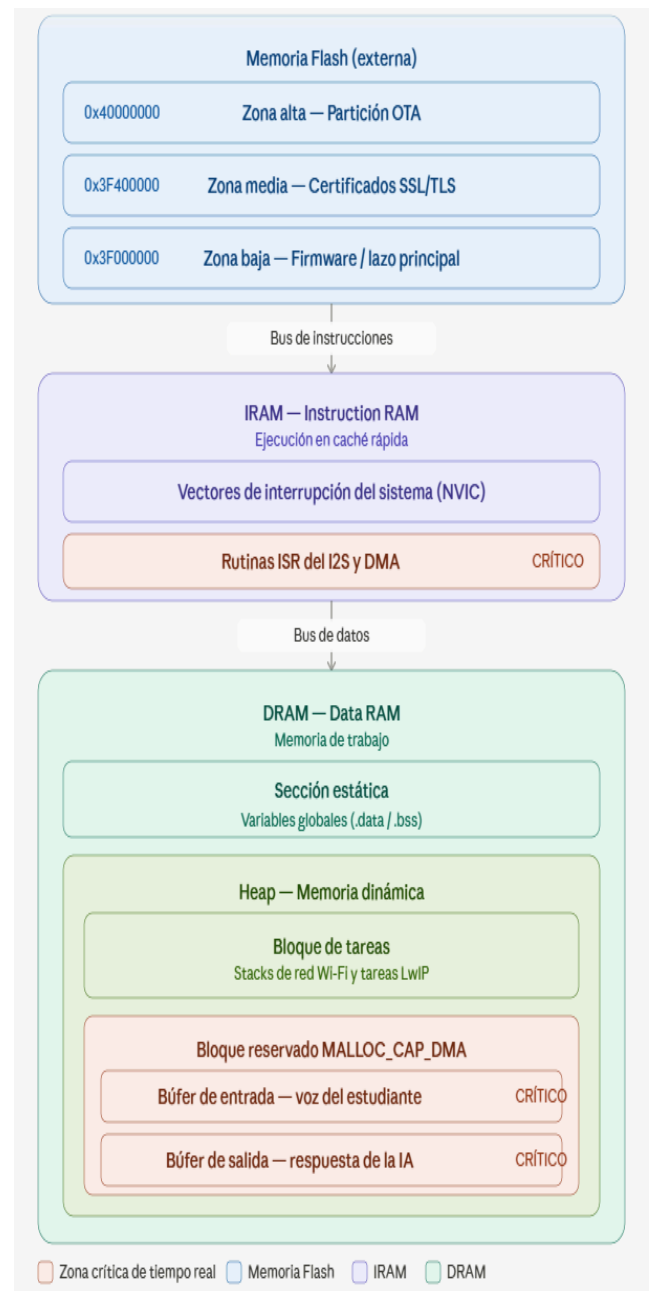
III. MAPA DE MEMORIA Y ORGANIZACIÓN DE DATOS

El SoC ESP32 se rige por una arquitectura Harvard modificada. La separación física de los buses de instrucciones y datos previene colisiones al manejar flujos masivos como el audio.

Tipo de Memoria	Función Arquitectónica en el Proyecto
Flash SPI (Externa) (4 MB a 16 MB)	Almacena el código binario compilado (firmware estático), certificados SSL de la red y rutinas de inicialización lentas.
IRAM (Instruction RAM) (192 KB)	Caché interna de cero estados de espera. El firmware copia aquí las Rutinas de Interrupción (ISR) del I2S durante el arranque para garantizar que la CPU atienda el audio en nanosegundos.
DRAM (Data RAM) (328 KB)	Memoria de trabajo. Alberga el segmento .bss (variables globales) y el Heap . Aquí se reservan forzosamente bloques contiguos de memoria para los descriptores DMA del audio.
RTC Memory (16kb)	Memoria aislada que sobrevive a reinicios (Deep Sleep). Se usará para guardar contadores de uso del asistente y credenciales Wi-Fi sin desgastar la Flash.

Para el correcto funcionamiento del sistema de audio, el código exige a la DRAM la asignación de memoria con el flag `MALLOC_CAP_DMA`. Si la red Wi-Fi fragmenta esta memoria (creando "huecos"), el hardware de audio no podrá

almacenar los datos continuos, lo que obligaría a un reinicio preventivo por parte del *Watchdog Timer*.



IV. INTERFACES DE COMUNICACIÓN Y ANÁLISIS A BAJO NIVEL

1. Configuración del Reloj en el Bus I2S: El protocolo I2S (Inter-IC Sound) requiere una sincronización de reloj estricta. Para configurar el hardware sin librerías opacas, el firmware modifica el registro `I2S_CLKM_CONF_REG` del ESP32. Para muestrear la voz del estudiante a una frecuencia estándar de 16 kHz con calidad de 32 bits en estéreo (requerido por el driver DMA), el reloj de bit (BCLK) generado por el microcontrolador se rige por la fórmula arquitectónica:

$$BCLK = 16000 \times 32 \times 2 = 1.024 \text{ MHz}$$

El ESP32 divide su reloj maestro interno de 160 MHz para alcanzar exactamente 1.024 MHz, garantizando que el micrófono INMP441 envíe cada bit en sincronía perfecta.

2. Pila de Red LwIP a Bajo Nivel: La transmisión hacia la Inteligencia Artificial se gestiona mediante *sockets* a nivel de sistema operativo. El núcleo Wi-Fi deposita los paquetes entrantes en una región de memoria protegida, disparando una interrupción al núcleo MAC. Esto permite separar físicamente el procesamiento de la antena de radio (en el Core 0) de la extracción del audio (en el Core 1).

